

Here is the finalized System Design and Technical Architecture Specification for **RepoCodex**. This version fully integrates the core graph engine, the scalable performance model, and the MCP Context API designed specifically for autonomous coding agents.

System Design & Architecture Specification: RepoCodex

Executable Repository Memory Framework

1. Executive Summary & Core Philosophy

Modern AI coding agents and human developers excel at writing syntax but struggle with institutional context. Repositories currently lack **memory**—the business rules, architectural constraints, and technical compromises that explain *why* code exists.

RepoCodex is an open-source, repository-native framework that functions as **Executable Repository Memory**. Instead of operating as a monolithic static analyzer, it acts as an orchestration layer. It utilizes a **Bi-Partite Graph** to connect human/AI intent to source code, validates constraints deterministically using existing community engines ([ast-grep](#), [Tree-sitter](#)), and serves highly scoped context to autonomous agents and CI/CD pipelines via the Model Context Protocol (MCP).

2. The Bi-Partite Graph Model

RepoCodex solves the context gap by strictly isolating syntax from intent, connecting them via "Virtual Edges."

1. **Layer 1: The Code Graph (Syntax & Structure):** Parsed dynamically using [Tree-sitter](#) and [SCIP](#). Represents the physical codebase geometry.
2. **Layer 2: The Intent Graph (Memory):** Stored natively in the repository inside the [.context/](#) folder using **Google's Open Knowledge Format (OKF)**. Since it lives alongside the code, it branches, merges, and rolls back atomically.
3. **Virtual Edges (Executable Contracts):** Directional links defined in OKF YAML ([entry_points](#)). These map Layer 1 to Layer 2 and are verified via [ast-grep](#) structural queries.

3. Memory Schemas & Authoring ([.context/](#))

Developers and agents author memory in the [.context/](#) directory exactly as they write code. The framework supports two primary schemas enforced via Pydantic models:

A. Business & Architectural Invariants (Strict Validation)

Rules that can be deterministically verified against the AST to block regressions.

YAML

```
id: INV-802
type: InvariantContract
lifecycle:
  state: ACTIVE
provenance:
  author: "@human-dev"
entry_points:
  - "src/billing/PaymentGateway.ts"
verification:
  strategy: AST_RULE
  engine: ast-grep
  rule: |
    pattern: |
      if ($ACCOUNT.tier == ENTERPRISE) { $GRACE = 3; }
```

B. Technical Decisions (Contextual Anchors)

Implementation explanations or design choices that act as vital reading material for AI agents before they modify code.

Markdown

```
id: TECH-042
type: TechnicalDecision
lifecycle:
  state: ACTIVE
entry_points:
  - "src/core/streams/CustomDataStreamer.py"
```

Custom Data Streamer Implementation

****Agent Directive:**** Do not optimize this into a standard list comprehension.

Standard generators caused a severe memory leak during batch ingestion here because they hold references to the unparsed XML tree.

4. Scaling for Massive Monorepos (Performance Architecture)

To prevent the framework from choking on million-line repositories, RepoCodex utilizes **Lazy Evaluation and Scoped Boundaries**.

- **Diff-Driven Parsing ($O(\Delta)$):** The local engine never parses the full repository during active development. It uses `git diff` to identify modified lines, running `Tree-sitter` *only* on the uncommitted diff. Performance scales with PR size, not repository size.
- **High-Speed SQLite Indexing:** Virtual edges are indexed in a local, ephemeral `.cache.sqlite` database. Querying the graph to find which OKF contracts intersect with a modified function takes ~2-5 milliseconds.
- **Targeted Validation:** RepoCodex does not run all 5,000 architectural rules against the codebase. If the diff touches the `billing` module, it retrieves and executes *only* the `ast-grep` rules associated with `billing` contracts.

5. Resilience: Auto-Healing Through Refactoring

When code moves, memory edges break. RepoCodex employs **Semantic Re-anchoring** to survive refactoring.

1. **1:1 Renames:** Detected natively via `git diff -M`. RepoCodex silently updates the OKF `entry_points` array to the new file path.
2. **1:N File Splits (AST Self-Healing):** If a file is split and an Invariant loses its entry point, RepoCodex runs the invariant's `ast-grep` rule against the newly created files. If it finds a match, it auto-heals the edge and updates the YAML.
3. **Ambiguous Deletions:** If an edge is severed and no AST match is found, RepoCodex intercepts the commit. Humans receive a terminal prompt; Agents receive a system directive forcing them to update the `entry_points` in the `.context/` file.
4. **Garbage Collection:** If a module is entirely deleted, the OKF contract transitions to `state: ORPHANED`, preserving history but dropping out of the active validation loop.

6. The Agent Context API (MCP Interface)

To support autonomous LLM loops without exhausting token limits, RepoCodex exposes the engine via the Model Context Protocol (MCP) using **Sub-Graph Extraction**.

MCP Tool	Purpose	Mechanism

repocodex_get_context	Pre-coding Context Injection	Queries SQLite to return only 1st and 2nd-degree OKF neighbors for the files the agent intends to edit, returning a highly scoped, token-efficient payload.
repocodex_validate_diff	Continuous Validation	Executes the ast-grep engine against the agent's uncommitted code. Returns PASS or a strict JSON FAIL with the broken ast-grep rule and OKF narrative.
repocodex_propose_mutation	Autonomous Memory Update	Allows the agent to rewrite a rule if a prompt requires a business logic change. Saves to .context/ with state: PROPOSED_BY_AGENT to ensure safe human review.

7. End-to-End Workflows

Workflow A: Autonomous Agent Loops (Non-Blocking)

1. **Context Fetch:** Agent calls **repocodex_get_context** before coding, injecting the institutional memory into its system prompt.
2. **Validation & Loop:** Agent modifies code and calls **repocodex_validate_diff**. It fails **INV-802**.
3. **Autonomous Mutation:** Realizing the rule must change to satisfy the user, the agent uses **repocodex_propose_mutation** to mutate **.context/INV-802.md**.
4. **Provenance:** RepoCodex safely marks the YAML with **state: PROPOSED_BY_AGENT**. The agent commits the code without terminal freezes, deferring final approval to the human PR reviewer.

Workflow B: Human IDE Workflow (Interactive gates)

1. **Interception:** Developer modifies code and types `git commit` (or uses IDE UI). The Agent Plugin 1.0 `PreToolUse` hook intercepts.
2. **Validation:** RepoCodex checks the diff and catches a violation locally (100ms).
3. **LLM Proposer:** The CLI passes a directive to the IDE's LLM, which prompts the user: *"You broke INV-802. Is this intentional?"*
4. **Atomic Update:** The human provides rationale. The LLM drafts the OKF update and, upon human `[Y]` confirmation, executes the atomic commit.

Workflow C: CI/CD Architecture Drift Detection

If code is pushed that violates an invariant without a corresponding `.context/` mutation, a stateless GitHub Action / App runs the headless RepoCodex container and hard-blocks the PR (Checks API: Failed), ensuring architectural drift never reaches `main`.

8. Implementation Stack & MVP Deliverables

The V1 Engine relies on Python-driven orchestration over compiled analysis tools, built for rapid open-source deployment:

- **CLI & Orchestrator:** Python (`typer`, `LangGraph`) for Git hook injection, state management, and terminal UX.
- **Schema Enforcement:** Python (`pydantic`) for strict OKF YAML validation.
- **Micro-diffing:** `tree-sitter` (C/Rust bindings).
- **Graph Cache:** `sqlite3` for sub-millisecond edge resolution.
- **Validation Engine:** `ast-grep-cli` (executed as a subprocess using community-standard structural queries).
- **IDE Integration:** Distributed as an **Agent Plugin 1.0** manifest (including the MCP server definitions), granting native integration into Claude Code, Cursor, and VS Code GitHub Copilot.

